The Journal of Open Source Software

BibLaTeX Package for Automating Bibliography Data for the APA Referencing Style

Amir Mohammad Tahsiri [1*]

1 Independent Researcher * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 27 February 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

This BibLaTeX-based LaTeX package named `APA-Custom-Template` offers a robust solution for APA (7th edition) referencing with numeric in-text citations, a format required by some academic journals. It ensures strict compliance with APA standards (based on this developer's tests current widely used APA packages lack this strict compliance) while minimizing manual effort through intelligent automation of unorganized bibliography entries. This package is compatible with MikTeX for Windows users and other LaTeX distributions.

What distinguishes this package from existing LaTeX solutions for APA referencing is its capacity to manage unorganized or missing entries in the bibliography data. It automatically extracts essential information from bibliography entries and formats it according to APA guidelines. Furthermore, it distinguishes between variations of entry types—such as proceedings papers (presented or published in journals, or books)—all identified by `@inproceedings`, using targeted checks. These features are especially valuable for users managing large reference lists who need precise formatting.

Statement of Need

LaTeX users typically integrate references into their documents by downloading BibTeX citations and adding them to a `.bib` file. However, these entries are often incomplete or unorganized: a single field may contain data for multiple fields, while others may omit critical information. For instance, a proceedings paper (`@inproceedings`) from DBLP—a reputable and widely used source—provides comprehensive bibliography data, often more complete than official publisher sites. Yet, its `booktitle` field typically bundles information for other fields, such as the conference `location` and `date`.

To this developer's knowledge, even the most comprehensive existing APA referencing packages, such as the standard BibLaTeX APA package (plk, 2010) or Springer Nature's BibTeX package (godkingjay, 2023), lack automated identification and extraction of bundled fields to reorder them correctly. Instead, they print the entire contents of each field consecutively. This developer's tests reveal that these packages also output verbatim lengthy fields, such as `abstract` or `note` fields. Moreover, even when fields like `location` or `date` are separate, existing tools fail to recognize them. In regards to recognition, existing packages even fail to recognise critical fields for certain entries. For example, suppose a proceedings paper published in an edited book lacks a critical field like `editor`. In that case, current packages do not flag the issue or prompt users for corrections, simply reproducing the entry unchanged.

Another critical need the `APA-Custom-Template` addresses is its ability to differentiate subtypes of a specific entry type. For instance, it can distinguish whether a proceedings paper is a presentation, published in a journal, or included in a book, despite all sharing the `@inproceedings` identifier. Without this package, users would need to manually identify these distinctions.

Unlike other tools, this package automates this process. Additionally, when missing fields render subtype recognition impossible, requiring manual retrieval from online sources, the `APA-Custom-Template` ingeniously relies on easily accessible fields—like the URL—to distinguish between entry subtypes, simplifying the process for users.

Beyond outperforming other packages, this solution leverages powerful and fundamental techniques like `regex` commands and the `expl3` package designed for robust list manipulation and iteration. In addition, the package's innovative methods prioritize both efficiency and accuracy. The use of fundamental commands, powerful packages, and innovative methods embody a high flexibility within this package making it adaptable to other referencing styles. With minor to moderate modifications, this package can become used for referencing styles such as Harvard or Chicago etc, broadening its utility for diverse academic needs.

Implementation

The main algorithms that make this package unique and require explanation are the macro `\ExtractFieldInfo` and the bibliography driver for `inproceedings` located in the `APA-Custom_LaTeX-Template.bbx` file, explaining the extraction of bundled fields and differentiating between subtypes of an entry respectively.

The ExtractFieldInfo macro:

The `\ExtractFieldInfo` macro (line 246) takes an input field and splits it into a sequence of items based on commas (lines 248-249). The main processing loop `\l_my_continue_bool` then iterates through this sequence from right to left (lines 258-373). This direction for iteration is a delicate matter: since the conference title may contain commas as punctuation (not separators). Starting from the right ensures that the title is identified correctly after other fields (e.g., `date` and `location`) are extracted. By prioritizing the right-most items, the macro avoids misinterpreting the commas that are parts of the title as separate fields.

At the start of the loop (line 264), a regex checks the last item in the sequence for a specific token string (pattern): a four-digit string (i.e., a year) preceded by at least one non-digit character `{.\d{4}\Z}`[1]. If this token string is found at the right-most item in the list, it means the year is embedded within the title (e.g., "title 2025") and that all items in the list belong to the `title` of the paper. The macro wraps this year (and its acronym) in parentheses and sets the remaining items as the paper `title` (lines 267-276). Such a token string is considered unique amongst other possible list items (i.e., `stand-alone-year`), and by taking advantage of this item's unique token string, the macro can avoid unnecessary searches through the entire list, efficiently isolating the title and saving computational effort.

If the last item does not match this pattern, the macro checks if the item in the list matches the string "proceedings" (lines 282-287). If so, it discards (pops) this item, as it is not relevant to the title, date, or location. Next, the macro uses another regex to check for a `stand-alone-year` `{^\d{4}}`[2] (with no preceding characters) in the current item (line 289). If found, the year is discarded (line 293), and the next item is checked for a month (e.g., "January") to store in the date variable (lines 297-302). However, if a `stand-alone-year` is not found, the macro does not check for other items and returns to the loop start as the title is the only remaining item.

If a date is identified, the next item(s) are assumed to be the location. However, location fields can vary: they typically include two items (city and country), but if the country is 'USA', a third item (state) is present. To handle this, the macro checks if the first location item is 'USA' (line 305). If so, it uses a nested while loop to extract the next two items (city and

[1] Token "d" represents a digit, d4 means a four-digit string, and the "." before means preceded by at least one non-digit character.

[2] Same as before, exept that ^ means no preceding characters.

state); otherwise, it extracts only the next item (city) (lines 305-328). These items are stored in a temporary sequence `\l_my_location_list_seq` from left to right to preserve (first out, first in) their original order (line 302). The nested loop ensures all location items are accounted for before exiting (lines 330-338).

After processing the location, the macro returns to the main loop and rechecks for the `embedded-year` pattern (line 258). If no special conditions remain, it builds the title by adding items to `\g_my_title_tl` variable from left to right, separating them with commas and spaces (lines 354-359). This ensures the title is formatted consistently, even if it contains internal commas. Finally, the macro checks if the input sequence is empty to exit the loop or continue processing (lines 362-372).

The inproceedings bibliography driver:

Regarding the bibliography driver for `inproceedings`. The driver examines whether the editor field is undefined. If so, it searches for the `biburl` field. If that is undefined, it looks for an `isbn` field. In the rare case that none of these fields are defined, it prints a message prompting the user to retrieve the `biburl` from the URL (address bar) of its online source (lines 485-490). The address bar almost always contains a specific key word identifying the type of paper.

If the entry includes an `isbn` field, we can confirm the proceedings paper is part of an edited book as this field is unique to published books. In this case, the driver proceeds to execute the `EditorMode` function (line 491). The `EditorMode` function formats the output for references within an edited book. A potential question arises: if the `editor` field is undefined at this stage, what does it mean to execute a function designed for printing the editor field? The `EditorMode` function addresses this by outputting *"Find editors from publisher's source!"* if the editor field is absent.

Next, the driver handles cases where the `biburl` field is defined. It employs a developer-defined function, `ExtractURLInfo`, which uses a regex command to scan for terms like *"article"* or *"journals"*. If a match occurs, we can confidently classify the paper as a journal article where the code prompts the user *"This paper is an article, find the correct BibTeX reference!"* (lines 495-503). If no match is found—indicating terms like `conference`, `proceedings`, or `chapter`—and the editor field is absent (with incomplete data already accounted for), the paper is definitively a paper presentation (lines 505-514).

The driver then addresses scenarios where the editor field is defined. It checks if both `biburl` and `isbn` are undefined. If so, it prints *"Find biburl from online source!"* However, if `isbn` is defined, we can confirm the paper belongs to an edited book, and the driver prints using `EditorMode` (lines 522-525). When `biburl` is defined, the process mirrors the earlier logic (lines 527-534). Although, this time if the paper is not a journal article, it is part of an edited book, and `EditorMode` is applied (line 532).

Summary of Key Innovations

Key innovations of ExtractFieldInfo macro:

- Splitting the input into a sequence variable (`\l_my_input_seq`) using commas as separators.
- Iterating from **right to left** to prevent misinterpreting commas within the title as item separators.
- Utilizing a unique regex identifier for an `embedded-year` (e.g., "title 2025") to potentially eliminate additional conditional checks, enhancing macro efficiency.
- Popping the sequence's first item to check for a possible `proceedings` item.
- Skipping checks for `date` and `location` if no `stand-alone-year` is present, returning to the loop's start since the remaining items belong to the `title`.

- Rebuilding the title in the variable `\g_my_title_tl` from scratch (left to right) if no `embedded year` is found. The title cannot just be stored in the variable (similar to when `embedded-year` exists) due to the first item being 'popped' from the sequence (meaning popped from `title`) if `proceedings` or `stand-alone-year` were not found.

Key innovations of inproceedings bibliography driver:

- Purposely conducting checks with disparate fields (e.g., `isbn` check when no `editor` present or *"journal"/"article"* check when `editor` present) to detect missing critical fields or excessive ones, prompting the user to ensure the omission or addition is addressed.
- Deliberately using the `isbn` and `biburl` fields to mitigate issues caused by potentially missing data in identifying subtypes of proceeding papers, as it is improbable that both fields are absent.
- Using the uniqueness of the `biburl`, as it reliably includes a clear key identifier of the paper type.
- Should the user have to supply missing fields, the `biburl` field (from the online address bar) is the easiest to provide, aligning with the package's automation goal.

References

godkingjay. (2023). *Springer-nature-latex-template*. https://github.com/godkingjay/springer-nature-latex-template.git.

plk. (2010). *Biblatex-apa*. https://github.com/plk/biblatex-apa.git.

DRAFT